Width, batch and horizon: a conditional hyperparameter-transfer protocol

Define the update before naming its learning rate

A base LR is only one number in a training implementation. Record parameterization, initialization, optimizer equations, actual parameter-group LRs, decay exclusions, epsilon, clipping, precision and schedule. Under µP, some groups scale differently with width; copying a scalar LR between implementations can change the effective update substantially.

Tensor Programs V, Step Law v3 and Power Lines v2 address related but different conditional questions. The Marin optimizer-search manual traces its code and actual multiplier grids. None supplies a universally validated optimizer for our recurrent MoE.

Make a parameter-group contract

For every parameter group, save tensor selector/name pattern, shape role, actual peak/final LR, decay, epsilon, momentum/betas, clipping/reduction convention and optimizer-specific normalization. Emit the resolved group table from the trainer rather than inferring it from a high-level config.

Group	Questions that must be answered
Input embedding / output head	Tied or untied? Initialization and readout scaling? Padded vocab? Actual LR?
Attention / dense matrices	µP width factors? Fan-in convention? Per-matrix norm or trust region?
Routed experts	Which width is scaled: residual, per-expert hidden or expert count? Shared normalization?
Router / biases / norms	Separate optimizer/LR/decay? Auxiliary-loss and clipping treatment?
Reused blocks	Summed gradients across visits? Visit-count normalization? Intermediate losses?

These are invariants to expose, not recommendations to force a particular group split. MuonH, AdamH and AdamW are different update rules. A group-wise convention validated for one does not automatically apply to another.

Separate three transfer experiments

1. **Width transfer:** keep depth, family, data, context, token horizon and declared batch/schedule controls fixed; test whether the near-optimal LR region persists as width changes.
2. **Shape transfer:** change depth/aspect ratio under a declared budget; retune or validate depth-sensitive initialization, residual scaling and optimizer settings. A fixed-depth width ladder cannot answer this question.
3. **Horizon/batch transfer:** change tokens and batch with update-count/warmup/annealing semantics specified. Fit a conditional law only after measuring optima in independently varying dimensions.

The sources support distinct evidence scopes. Claim `claim-mup-scope` does not promote width transfer into depth transfer. Step Law's non-power-of-two widths establish a usable candidate set, not the optimal shape for our mesh.

Grid design and a concrete empirical anchor

For each selected size/horizon cell, bracket LR and token batch broadly enough to locate an interior low-loss region. Include failed/unstable and boundary-optimal cells in the record. Use a coarse grid, narrow around promising regions, repeat the finalists, and reserve an out-of-sample scale. This is our search protocol, not a claim that every lab used exactly this sequence.

The released Step Law dense cohort includes a1,073,741,824-parameter/20B-token group with118observations. The locally replayed best observed smooth loss is2.2254960 at LR0.001381 and256sequences×2048=524,288tokens per batch. Replay receipt. This is a finite-grid observation under the released recipe, not a continuous optimum or an externally validated hero LR.

```
python scripts/kb_workbench.py step-law-release
```

The grouping includes declared shapes and horizon and, for MoE, exposed expert/routing fields. Missing recipe metadata can still confound interpretation. The release contains1911dense and708MoErows, which is a different scope from the paper's overall training count.

Batch, momentum, decay and horizon are coupled

Let token batch be B_t, horizon D and update count $S \approx D/B_t$. An optimizer memory length of about $1/(1-\beta)$ updates corresponds to about $B_t/(1-\beta)$tokens. Holding beta fixed while increasing batch changes the token-scale memory. A warmup defined in updates likewise covers a different number of tokens. These accounting facts do not establish an optimal beta scaling law.

For constant actual group LR η and decoupled decay λ, the gradient-free recurrence is

$$W_{s+1} = (1-\eta\lambda)W_s, \qquad \tau = \frac{B_t}{\eta\lambda D}.$$

The Power Lines account studies empirical optimal timescales in its own family. Its finding is not "keep τ constant across every model and horizon"; the fitted optimum changes with regime.

```
python scripts/kb_workbench.py optimizer-timescale
```

Our fixture doubles batch and compares an approximate doubled decay with the exact decay preserving total multiplicative shrinkage. It has zero gradient and constant LR. It tests the accounting and numerical implementation, **not** an AdamW convergence claim or a MuonH prescription. During annealing, cumulative $\sum_s \eta_s\lambda_s$ and the exact product are more informative than one nominal peak value.

Selection and extrapolation

Rank matched, fully annealed endpoints on the fixed evaluation contract. Repeated trials and shared-prefix branches need their own variance treatment. Fit the selected HP law in declared units, inspect held-out residuals and propagate uncertainty into the hero search bracket. A paper coefficient can center a search; it cannot qualify our data/tokenizer/optimizer/recurrence changes by itself.

Reject transfer if the candidate optimum remains on the bracket boundary, if stability degrades, if resolved group conventions differ, or if a second size/horizon fails the prediction beyond the predeclared tolerance. Preserve the rejected run and diagnostic reason. The next step is an identifiable fit and untouched holdout, not a larger launch justified solely by a neat power law.